

✓ 5 Supervised Learning

Chapter Introduction

Chapter 5 introduces the fundamentals of machine learning.

Goal: To learn the structure and overview of machine learning and to gain the ability to correctly build and evaluate models.

- **5.1 Overview of Machine Learning**

- 5.1.1 What is Machine Learning?
- 5.1.2 Supervised Learning
- 5.1.3 Unsupervised Learning
- 5.1.4 Reinforcement Learning
- 5.1.5 Importing Libraries for This Chapter

- **5.2 Multiple Regression**

- 5.2.1 Importing Car Price Data
- 5.2.2 Data Preparation
- 5.2.3 Model Building and Evaluation
- 5.2.4 Summary of Model Building and Evaluation Process

- **5.3 Logistic Regression**

- 5.3.1 Example of Logistic Regression
- 5.3.2 Data Preparation
- 5.3.3 Model Building and Evaluation
- 5.3.4 Improving Prediction Accuracy with Scaling

- **5.4 Regularized Regression: Lasso and Ridge**

- 5.4.1 Features of Lasso and Ridge Regression
- 5.4.2 Comparison of Multiple and Ridge Regression

- **5.5 Overfitting**

- **5.6 Decision Trees**

- 5.6.1 Mushroom Dataset
- 5.6.2 Data Organization
- 5.6.3 Entropy: Impurity Measure
- 5.6.4 Information Gain: Evaluating Split Value
- 5.6.5 Building a Decision Tree Model

- **5.7 k-Nearest Neighbors(k-NN)**

- 5.7.1 Building a k-NN Model

- **5.8 Support Vector Machine**

- 5.8.1 Building a Support Vector Machine Model

- **5.9 Comprehensive Questions**

- ■ Comprehensive Question 5-1 Terms in Supervised Learning (1)
- ■ Comprehensive Question 5-2 Decision Trees
- ■ Comprehensive Question 5-3 No Free Lunch

✓ 5.1 Overview of Machine Learning

Goal: Understand the structure of machine learning and gain an overview of supervised and unsupervised learning.

In this chapter, we will learn about specific methods in supervised learning. Through this chapter, we aim to understand the principles of machine learning, along with the basics of model building, and gain the skills to implement them accurately.

Before diving into supervised learning, let's first take a broad look at the entire landscape of machine learning.

✓ 5.1.1 What is Machine Learning

Keywords: Machine Learning, Supervised Learning, Unsupervised Learning, Reinforcement Learning

Machine Learning (machine learning) is a technology that enables machines to acquire knowledge or behaviors to achieve specific objectives by processing data. Machine learning is broadly categorized into **supervised learning**, **unsupervised learning** and **reinforcement learning**. Alternatively, it can be divided into two main types--supervised and unsupervised learning--or even four types by adding semi-supervised learning to the previous three categories.

✓ Supervised and Unsupervised Learning

The data used to enable machines to acquire knowledge or behavior is called training data. The difference between supervised and unsupervised learning lies in whether the training data includes target variables and explanatory variables (discussed later). In short, supervised learning involves providing labeled data with correct answers, while unsupervised learning does not.

1. Supervised Learning

This method seeks to build a model that predicts an output variable (target) from input variables (features). In supervised learning, the training data includes both target and explanatory variables. The model's parameters are adjusted so that it best approximates the target variables using the explanatory variables of the training data as inputs. This process will be explained in detail in this chapter.

For example, supervised learning can be used to identify whether an email is spam based on its title and content (explanatory variables) or to predict stock prices based on trading activity (explanatory variables).

2. Unsupervised Learning

Unsupervised learning focuses on the input data itself, aiming to discover patterns or insights hidden within the data. In this approach, training data does not contain target or explanatory variables. Techniques include clustering, which groups a large number of data points into similar clusters, and Principal Component Analysis (PCA), which reduces the dimensionality of the data while preserving essential information. These methods are often used in exploratory data analysis to interpret data and in dimensionality reduction.

Reinforcement Learning

Reinforcement learning is a method that enables machines to acquire knowledge or behavior rules by providing rewards based on the outcomes of a series of actions taken by the program (machine). In supervised learning, it is necessary to assign correct data to each individual action; however, in interactive environments, such as games with opponents, it is challenging to provide correct data for every possible situation.

Consequently, reinforcement learning techniques, which allow machines to learn behavior rules based on rewards from the results of a series of actions, have gained attention in recent years as a learning approach that can handle cases difficult to express in supervised learning.

✓ Applying Machine Learning

In this book, we will begin by applying machine learning to the provided data. However, in actual data analysis scenarios, it is essential to conduct basic observations and understand the data, such as obtaining descriptive statistics and creating histograms or scatter plots. The quality of the data significantly impacts the quality of the machine learning outputs. Moreover, these initial checks can lead to valuable insights about the data. Remember that the goal is not merely to use machine learning; it is just one tool among many in the data analysis process.

[Key Points]

When conducting data analysis in practice, before applying machine learning, create descriptive statistics and scatter plots to grasp the trends and overall picture of the data.

For beginners in machine learning, references such as the "Books Useful for Machine Learning with Python" in the bibliography may be helpful.

To learn how to leverage machine learning from a business perspective, references such as the "Books Useful for Applying Machine Learning from a Business Perspective" in the bibliography can be beneficial.

Additionally, it is impractical for a single specialized book to cover all information regarding machine learning models and implementations. Therefore, if you encounter any uncertainties (such as parameter settings), it is essential to refer back to the official documentation. While the official documentation can be dense and challenging to read in its entirety, it provides reliable explanations for detailed parameters and other critical information.

[Key Points]

If you have questions about parameters or the characteristics of models such as machine learning, start by consulting the official documentation.

✓ 5.1.2 Supervised Learning

Keywords: Target Variable, Explanatory Variable, Regression, Classification

Supervised learning is a method for building models that predict the correct output based on given training data. As previously mentioned, the variable we want to predict within the training data is called the **target variable** (also known as the ground truth, response variable, target variable, or dependent variable), while the variables used to explain the target variable are referred to as **explanatory variables** (also known as features, predictor variables, or independent variables).

If we consider a function $y = f(x)$, x represents the explanatory variables, y represents the target variables, and the function $f(x)$ is the model. For example, if we want to predict whether a buyer of a certain consumer brand will become a non-buyer in the future (target variable), we would use various historical data (such as customer attributes, purchase frequency, and purchases of related brands) as explanatory variables.

✓ Supervised Learning Techniques

Supervised learning can be classified into several types based on the format of the target variable. When the target variable takes numerical values, such as stock prices, it is referred to as **regression**. In contrast, when the target variable is categorical, such as "male or female" or "toddler, elementary school student, student, adult," it is called **classification**. For example, in the case of predicting whether a buyer will become a non-buyer of a brand, this is a classification task that divides the outcomes into two categories: "will purchase" or "will not purchase."

Supervised learning algorithms include **multiple linear regression, logistic regression, k-Nearest Neighbors (k-NN), Decision Trees, Support Vector Machines (SVM), Random Forests, and Gradient Boosting**, among others. It's important to note that these techniques can be used for both regression and classification tasks.

For instance, although logistic regression has "regression" in its name, it is primarily used for classification purposes. Decision Trees are commonly used for classification tasks, referred to as classification trees, while for regression tasks, they are called regression trees. Each of these methods will be explained in detail later.

The choice of which method to use is fundamentally determined by the desired performance of the model. However, if interpretability of the learning results is prioritized, simpler methods such as multiple linear regression, logistic regression, or Decision Trees may be intentionally selected. This is because techniques like Support Vector Machines can be difficult to explain and are not easily understood by non-experts after a single explanation (though Decision Trees are often considered understandable, it's important to note that they may not be intuitive for all non-experts).

It is essential to assess whether the situation demands a focus on interpretability or if precision should be prioritized, and make decisions on a case-by-case basis.

✓ 5.1.3 Unsupervised Learning

Keywords: Clustering, Principal Component Analysis, Market Basket Analysis

Unsupervised learning focuses on the input data itself, aiming to discover patterns or insights hidden within the data, without the presence of a target variable.

✓ Unsupervised Learning Techniques

A representative method of unsupervised learning is **clustering**, which groups a large number of data points into similar clusters. For example, it can be used in marketing analysis to identify which preference groups consumers belong to.

Clustering is positioned as an exploratory data analysis technique, as it seeks to uncover the characteristics of the data itself. The process does not end with grouping the data based on clustering results; it is crucial to interpret these results and verify that they align with business insights or practical understanding. Complete automation in exploratory data analysis is challenging, and human judgment plays a vital role in this process.

Other methods of unsupervised learning include Principal Component Analysis (PCA) and Market Basket Analysis.

Principal Component Analysis is a technique used to reduce the number of variables by condensing multiple variables into fewer ones while preserving the information.

Market Basket Analysis is applied to purchasing data, often referred to as Point of Sales (POS) data, to discover association rules, such as identifying that people who purchase product A are highly likely to also purchase product B. This method reveals combinations of events that are strongly correlated.

In the collection of references listed, books on data mining categorize supervised learning as "goal-oriented data mining" and unsupervised learning as "exploratory data mining." These resources provide insights into how machine learning and data mining are utilized in business contexts. They are recommended for those who wish to deepen their understanding of the material from a business perspective. Additionally, it is worth noting that the top two books in the reference collection are translations that have omitted parts of the original texts, so those who can read English may prefer to refer to the original editions.

✓ 5.1.4 Reinforcement Learning

Keywords: Dynamic Programming, Monte Carlo Methods, Temporal Difference Learning (TD Learning)

Reinforcement learning is a technique that enables machines to learn behavior rules to maximize a reward. The rewards are designed to align with the objectives, meaning desirable outcomes receive high rewards, while undesirable outcomes receive low

rewards. Unlike supervised learning, where correct data is provided for each action, reinforcement learning seeks to discover which actions will ultimately yield greater rewards. In this framework, learning occurs within an environment where the machine (agent) interacts with other agents and the surroundings.

For example, in reinforcement learning, consider a scenario where a "baby (agent)" learns to walk by trial and error without being explicitly taught how to do so within its environment. Similarly, an "automobile (agent)" learns to navigate without colliding with other vehicles (other agents). These instances illustrate the principles of reinforcement learning.

✓ Reinforcement Learning

In reinforcement learning, a key theme is how to handle the exploration-exploitation dilemma. Since the agent learns through exploratory actions and interactions with the environment, balancing these two aspects is crucial. If the agent focuses solely on taking the "best known action" based on past experiences, it may miss out on discovering new actions (leading to knowledge bias). Conversely, if the agent excessively seeks "better actions" by trying new actions constantly, it may fail to utilize valuable past experiences (leading to exploration bias). Finding the right balance between exploration and knowledge utilization is essential for effective learning.

Reinforcement learning approaches include dynamic programming, Monte Carlo methods, and TD learning. Dynamic programming assumes that explicit knowledge is available, while Monte Carlo methods require only experience and do not necessitate complete knowledge of the environment. This book will only introduce the concepts of reinforcement learning. To learn more, see terms above, the "Books for Learning Reinforcement Learning," or the [OpenAI website](#).

✓ 5.1.5 Importing Libraries Used in This Chapter

In this chapter, we will use various libraries introduced in previous lectures, as well as the machine learning library Scikit-learn.

Scikit-learn was also used during simple linear regression analysis. As mentioned above, the official documentation for Scikit-learn at the reference URL "B-27" provides detailed

specifications and usage instructions, so please refer to it. The Scikit-learn library includes not only classes for machine learning but also several sample datasets.

In this chapter, we will proceed with the assumption that the following libraries have been imported.

```
# Data manipulation, processing, and analysis libraries
import numpy as np
import numpy.random as random
import scipy as sp
from pandas import Series, DataFrame
import pandas as pd

# Visualization libraries
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
%matplotlib inline

# Machine learning library
import sklearn

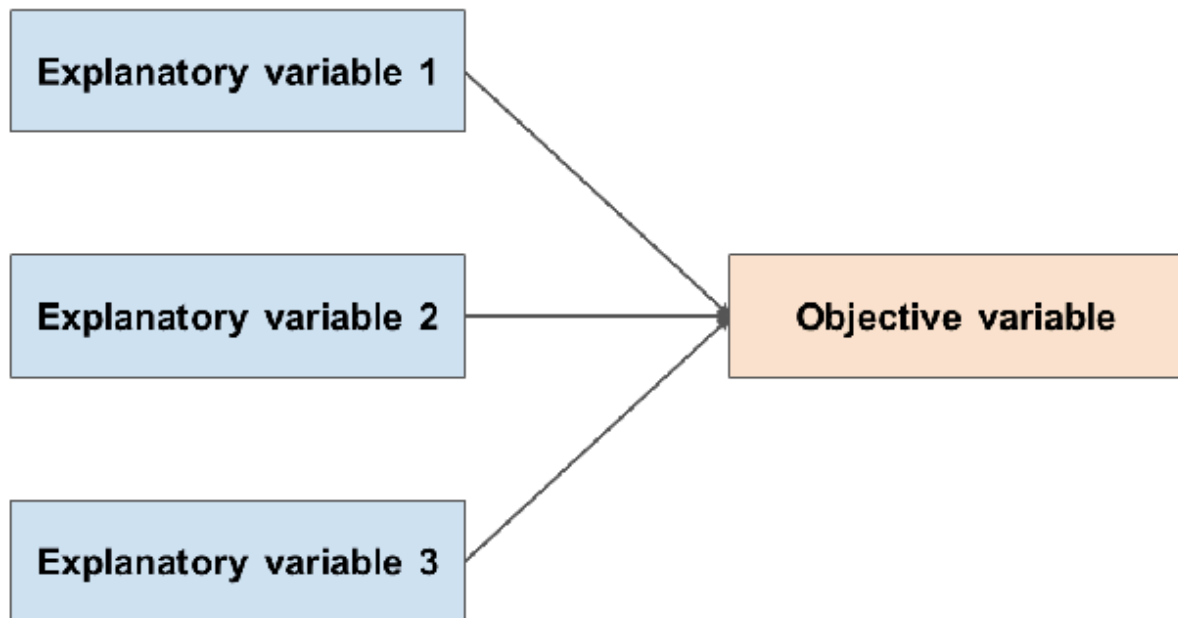
# Display up to three decimal places
%precision 3

'%.3f'
```

✓ 5.2 Multiple Regression

Keywords: Target Variable, Explanatory Variables, Multicollinearity, Variable Selection Methods

As the first topic in supervised learning, we will start by learning about **multiple linear regression**. In simple linear regression, there was only one explanatory variable for the target variable. Multiple regression extends this concept to cases where there are multiple explanatory variables. Through multiple regression, the coefficients of each explanatory variable (regression coefficients) can be estimated, allowing for the calculation of predicted values. The regression coefficients are estimated in such a way that the squared error between the predicted values and the target variable is minimized. Below is a diagram illustrating multiple regression.



Reference URL: <http://diary-ba.up.n.seesaa.net/diary-ba/image/E9878DE59B9EE5B8B0E5BC8FE383A2E38387E383AB.png?d=a1>

✓ 5.2.1 Importing Automobile Price Data

Let's try it out in practice. Here, we will build a multiple regression model to predict automobile prices based on their attributes (such as size) using a dataset that contains the prices and attributes of automobiles. The data will be sourced from the following URL.

<http://archive.ics.uci.edu/ml/machine-learning-databases/autos/imports-85.data>

```
# Import
import requests, zipfile
import io

# Fetch automobile price data
url = 'http://archive.ics.uci.edu/ml/machine-learning-databases/autos/imports-85.data'
res = requests.get(url).content

# Load the fetched data into a DataFrame object
auto = pd.read_csv(io.StringIO(res.decode('utf-8')), header=None)

# Set labels for the data columns
auto.columns = ['symboling', 'normalized-losses', 'make', 'fuel-type', 'aspiration', 'body-style', 'drive-wheels', 'engine-location', 'wheels', 'price']
```

```
'curb-weight', 'engine-type', 'num-of-cylinders', '
'stroke', 'compression-ratio', 'horsepower', 'peak-
```

When the above program is executed, the variable `auto` will contain the automobile price data as a Pandas `DataFrame` object. Let's actually check what kind of data it contains.

```
print('Shape of the car data:{}'.format(auto.shape))
```

```
Shape of the car data:(205, 26)
```

It can be confirmed that the data consists of 205 rows and 26 columns.

Next, let's display the first five rows using the `head()` method.

```
auto.head()
```

	symboling	normalized- losses	make	fuel- type	aspiration	num- of- doors	body- style	drive- wheels	1
0	3	?	alfa- romero	gas	std	two	convertible	rwd	
1	3	?	alfa- romero	gas	std	two	convertible	rwd	
2	1	?	alfa- romero	gas	std	two	hatchback	rwd	
3	2	164	audi	gas	std	four	sedan	fwd	
4	2	164	audi	gas	std	four	sedan	4wd	

In this dataset, the price of the cars is set in the `price` column. Since we are trying to create a model to predict the price based on the car's attributes, the challenge is to build a model that predicts `price` using the values from the other columns.

To simplify the task, we will only use three explanatory variables: `horsepower`, `width`, and `height`. In other words, we will create a model that predicts the price variable based on the explanatory variables `horsepower`, `width`, and `height`.

✓ 5.2.2 Data Preparation

Input data may contain inappropriate or invalid entries. Therefore, the first step is to review the data and organize it appropriately.

✓ Removing Inappropriate Data

When reviewing the data using the `head()` method, you may notice that there are entries with '?'. Many machine learning algorithms only handle numerical data, so preprocessing is required to remove non-numeric data, such as '?'.

Since our goal is to predict `price` from `horsepower`, `width`, and `height`, we need to delete any rows containing '?' in these variables. Specifically, we will convert '?' entries to missing values and then remove the rows containing these missing values. The following code can be used to check how many '?' entries are present in the four variables we are working with: `horsepower`, `width`, `height`, and `rice`.

```
# Count the number of '?' in each column
auto = auto[['price', 'horsepower', 'width', 'height']]
auto.isin(['?']).sum()
```

	0
price	4
horsepower	2
width	0
height	0

dtype: int64

You can see that there are '?' data mixed into the `price` and `horsepower` columns, so we will use a technique we learned before in Pandas to exclude them. By doing the following, the rows containing '?' will be removed. When executed, you can confirm that the number of rows has decreased.

```
# Replace '?' with NaN and drop rows with NaN
auto = auto.replace('?', np.nan).dropna()
print('Shape of the car data:{}'.format(auto.shape))
```

Shape of the car data:(199, 4)

✓ Type Conversion

Let's check the data types here. You can do so using the following code.

```
print('Check data types (before type conversion)\n{}\n'.format(auto.dtypes))
```

```
Check data types (before type conversion)
price          object
horsepower     object
width          float64
height         float64
dtype: object
```

It can be confirmed that both price and horsepower are not numeric types. Therefore, we will use `to_numeric` to convert them to numeric types.

```
auto = auto.assign(price=pd.to_numeric(auto.price))
auto = auto.assign(horsepower=pd.to_numeric(auto.horsepower))
print('Data type check (after type conversion)\n{}\n'.format(auto.dtypes))
```

```
Data type check (after type conversion)
price          int64
horsepower     int64
width          float64
height         float64
dtype: object
```

✓ Correlation Check

With the above operations, all rows of the explanatory variables and the target variable have been processed to be free of missing values and in numeric data format. Next, we will check the correlation between each variable. You can use `corr()` as follows to check the correlation.

```
auto.corr()
```

	price	horsepower	width	height
price	1.000000	0.810533	0.753871	0.134990
horsepower	0.810533	1.000000	0.615315	-0.087407
width	0.753871	0.615315	1.000000	0.309223
height	0.134990	-0.087407	0.309223	1.000000

Since `price` is the target variable here, focusing on the other three variables, we notice that `width` and `horsepower` have a somewhat high correlation of around 0.6. This check is necessary because using highly correlated variables as explanatory variables in multiple regression can lead to **multi-collinearity**.

Multi-collinearity is a phenomenon where high correlation between variables causes an increase in the variance of regression coefficients, leading to a loss of coefficient significance. To avoid this, typically in multiple regression modeling, only a representative variable from groups of highly correlated variables is used. However, since this is an experiment, we won't be strict about this; both `width` and `horsepower` will be included along with `height` in building the model.

✓ 5.2.3 Model Building and Evaluation

Now that we have the data, let's build the model. The following program creates a multiple regression model and examines its performance.

In the program below, the explanatory variables are set as X and the target variable as y .

In machine learning model building, it's common to train the model using "training data" and then evaluate its performance by testing it with separate "test data" to see how accurately it performs on new data. Here, we use the `train_test_split` function from Scikit-learn's `model_selection` module to split the data into training and test sets.

This function randomly splits the data into two parts. The split ratio is determined by `test_size`. Here, with `test_size` set to 0.5, the data is divided equally (setting it to 0.4, for example, would split the data into a 4:6 ratio).

The `random_state` parameter controls the randomness level. Setting `random_state` to 0, as we do here, ensures that the split is consistent each time the code is run, eliminating randomness. If `random_state` isn't set to a fixed number, rows may be categorized into either training or test data differently with each execution, resulting in varying outcomes. In practical applications, fixing `random_state` to ensure reproducibility is very important.

The multiple regression model is built using the `LinearRegression` class. An instance is created with `model = LinearRegression()`, and the model is trained by loading the training data with `model.fit(X_train, y_train)`, completing the training process.

Once trained, you can check the coefficient of determination, regression coefficients, and intercept. The coefficient of determination indicates how closely the predicted values of the target variable match the actual values of the target variable.

The goal of machine learning is to achieve high generalization performance, meaning that the model can make accurate predictions even with unseen data. While it might seem ideal to focus on fitting the training data well, in reality, models often end up with high accuracy on training data but low accuracy on test data. This phenomenon is called **overfitting** or **excessive learning**, and it's crucial to check for overfitting during model building. We'll explore this topic in more detail in a later chapter.

In the objective function of multiple regression analysis, y represents the target value, and $f(\mathbf{x}) = w_1x_1 + w_2x_2 + w_3x_3 + \dots + b$ represents the predicted value. The goal is to find the coefficients that minimize the following equation.

$$\sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2$$

```
# Import for data splitting (training data and test data)
from sklearn.model_selection import train_test_split

# Import for building the multiple regression model
from sklearn.linear_model import LinearRegression

# Set `price` as the target variable and the rest as explanatory variables
X = auto.drop('price', axis=1)
y = auto['price']

# Split into training and test data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random

# Initialize and train the regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Display the coefficient of determination
print('Coefficient of determination (train): {:.3f}'.format(model.score(X_train,
print('Coefficient of determination (test): {:.3f}'.format(model.score(X_test, y

# Display regression coefficients and intercept
print('\nRegression coefficients\n{}'.format(pd.Series(model.coef_, index=X.co
print('Intercept: {:.3f}'.format(model.intercept_))
```

```
Coefficient of determination (train):0.733
Coefficient of determination (test):0.737

Regression coefficients
horsepower      81.651078
width           1829.174506
height          229.510077
dtype: float64
Intercept: -128409.046
```

The coefficient of determination can be obtained using the `score` method. From the results above, we can see that the score for the training data is 0.733 and for the test data is 0.737. Since the scores for both the training and test data are close, we can conclude that this model is not overfitting.

✓ 5.2.4 Summary of Model Building and Evaluation Process

This concludes the process of model building and evaluation using multiple regression. The same basic process will be followed for other models, such as decision trees and SVMs, which will be learned in the following sections. In other words, this flow is fundamental to model building and evaluation.

- Initialization of the class for various model constructions: `model = LinearRegression()`
- Splitting the data into explanatory variables and target variables: X and y
- Splitting into training and test data: `train_test_split(X, y, test_size=0.5, random_state=0)`
- Fitting the model using the training data (training): `model.fit(X_train, y_train)`
- Testing the model's generalization performance using the test data: `model.score(X_test, y_test)`

In this case, we arbitrarily selected three explanatory variables—`horsepower`, `width`, and `height`—for the model building, but there are several statistical methods for selecting variables. Specifically, methods such as **forward selection**, **backward elimination**, and **stepwise selection** are used. The criteria for selection include RMSE (Root Mean Squared Error), AIC (Akaike Information Criterion), and BIC (Bayesian Information Criterion). These methods are not universally effective, and factors such as the model's generalization performance and business domain knowledge are also

considered in the selection process. The details of these methods are omitted in this book, so those who wish to deepen their learning should look into them further.

[Try It Out]

Let's explore forward selection, backward elimination, and stepwise selection methods.

✓ <Practice Question 5-1>

We will use the car price data utilized in the main text. For this data, set the target variable as `price` and use `length` and `engine-size` as the explanatory variables to build a multiple regression model. When doing this, use `train_test_split` to split the data into training and test sets so that each set is half of the total data. Then, construct the model and calculate the model's score using the test data. When executing `train_test_split`, set the `random_state` option to 0.

```
# @title
# Set "price" as the target variable, and the others as explanatory variables
url = 'http://archive.ics.uci.edu/ml/machine-learning-databases/autos/imports-8
res = requests.get(url).content

auto = pd.read_csv(io.StringIO(res.decode('utf-8')), header=None)

auto.columns = [
    'symboling', 'normalized-losses', 'make', 'fuel-type', 'aspiration', 'num-of-dr
    'body-style', 'drive-wheels', 'engine-location', 'wheel-base', 'length', 'width'
    'curb-weight', 'engine-type', 'num-of-cylinders', 'engine-size', 'fuel-system',
    'stroke', 'compression-ratio', 'horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg
]

# Data preprocessing
auto = auto[['price', 'length', 'engine-size']]
auto = auto.replace('?', np.nan).dropna()

# Specify "price" as the target variable and the others as explanatory variable
## WRITE ME

# Split data into training and test sets
## WRITE ME

# Initialize and train the multiple regression class
## WRITE ME
```

```
# Display R^2 score  
## WRITE ME
```

> <Practice Question 5-2>

For the data from <Practice Question 5-1>, use the same target variable, `price`, but construct a multiple regression model using a different set of explanatory variables. Split the data so that half is used for training and half for testing, then calculate the model's score using the test data. When splitting the data into training and test sets, set the `random_state` function to 0. How has the model's result changed? Analyze and discuss the reasons for the change.

↳ 1 cell hidden

✓ 5.3 Logistic Regression

Keyword: Logistic regression, Cross-entropy loss function, Odds ratio

As mentioned earlier, a multiple regression model is a regression model with multiple explanatory variables, where the target variable is numeric. Such variables are called continuous variables.

The **logistic regression** learned in this section, on the other hand, deals with target variables that are not numeric. For example, it is used in scenarios such as predicting whether a product will be purchased or not, or whether a company will go bankrupt or not—these are categorical data. Variables in categorical form are called categorical variables.

The task of calculating the probability that a data sample belongs to a particular category is called classification, and one of the algorithms used for this task is logistic regression.

Although it has the name "regression," it is a classification algorithm, so be careful (also, it can be used not only for binary classification but also for multi-class problems with three or more categories). Unlike when the target variable is numeric, in classification tasks, the learning process minimizes the following objective function. This objective function is called the **cross-entropy error function**, and its value becomes smaller as the probability of predicting the correct category increases.

$$- \sum_{i=1}^n [y_i \log(f(x_i)) + (1 - y_i) \log(1 - f(x_i))]$$

✓ 5.3.1 Example of Logistic Regression

Now let's look at a specific example of logistic regression. In this example, we will construct a model to predict whether an individual's income exceeds \$50K (50,000 USD) based on personal data such as age, gender, and occupation. The source data can be obtained from the following URL:

<http://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data>

First, we will retrieve the data and set the column names as shown below. The data consists of 32,561 rows and 15 attributes, with no missing values. By using `head()`, you will see that the dataset contains a mix of categorical variables like `workclass` and `education`, along with numerical variables like age and `education_num`.

```
# Retrieve the data
url = 'http://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data'
res = requests.get(url).content

# Load the retrieved data as a DataFrame object
adult = pd.read_csv(io.StringIO(res.decode('utf-8')), header=None)

# Set column labels for the data
adult.columns = ['age', 'workclass', 'fnlwgt', 'education', 'education-num', 'marital-status',
                 'occupation', 'relationship', 'race', 'sex', 'capital-gain',
                 'native-country', 'flg-50K']

# Output the data's shape and missing values
print('Data Format:{}'.format(adult.shape))
print('Number of defects:{}'.format(adult.isnull().sum().sum()))

# Output the first 5 rows of the data
adult.head()
```

Data Format:(32561, 15)
 Number of defects:0

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relation
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-fa
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Hus

✓ 5.3.2 Data Preparation

In this dataset, the target variable indicating whether income exceeds `flg-50K`. The values in the data are `<=50K` and `>50K`, and since they are difficult to handle in this form, we will convert them into a variable with flags of 0 or 1. First, let's check how many rows contain `<=50K` and `>50K`.

```
adult.groupby('flg-50K').size()
```

```

      0
flg-50K
<=50K  24720
>50K   7841

dtype: int64
```

It can be seen that there are 24,720 rows with `<=50K` and 7,841 rows with `>50K`.

Next, add a column named 'fin_flg' and set a flag of 1 for rows with `>50K` and 0 for others. Use `lambda` or `map` to set the flags. After the conversion, check that the results match the previous aggregation.

```
# Add the 'fin_flg' column, and set it to 1 if the value of the 'flg-50K', otherwise
adult['fin_flg'] = adult['flg-50K'].map(lambda x: 1 if x == '>50K' else 0)
adult.groupby('fin_flg').size()
```

```

      0
fin_flg
0      24720
1       7841

dtype: int64
```

We can see that the number of rows for `<=50K` and `>50K` matches the number of rows for '0' and '1', indicating that the operation was successful.

✓ 5.3.3 Model Construction and Evaluation

Now, let's construct the logistic regression model. We will use the numerical variables `age`, `fnlwgt`, `education-num`, `capital-gain`, and `capital-loss` as explanatory variables. The target variable is the `fin_flg`, for which we previously set flags to "1" and "0."

For constructing the logistic regression model, we will use the `LogisticRegression` class. The methods for splitting the data into training and test sets and for evaluation with the `score` method are the same as those used in multiple regression.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Setting explanatory variables and target variable
X = adult[['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss']]
y = adult['fin_flg']

# Splitting into training and test data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random

# Initializing and training the logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

print('Accuracy (train): {:.3f}'.format(model.score(X_train, y_train)))
print('Accuracy (test): {:.3f}'.format(model.score(X_test, y_test)))
```

```
Accuracy (train):0.796
Accuracy (test):0.799
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469: C
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (`max_iter`) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

From the above results, we can see that the accuracy is approximately 80% for both the training and test data, indicating that overfitting has not occurred.

Let's check the coefficients for each variable (age, fnlwgt, education-num, capital-gain, capital-loss) in the trained model by retrieving the `coef_` attribute.

```
model.coef_
```

```
array([[ -5.624e-03,  -3.846e-06,  -3.642e-02,   3.286e-04,   7.666e-04]])
```

Additionally, the odds ratios for each can be calculated as follows. The odds ratio indicates how much each coefficient affects the accuracy when it increases by 1.

```
np.exp(model.coef_)
```

```
array([[0.994, 1.    , 0.964, 1.    , 1.001]])
```

✓ 5.3.4 Improving Prediction Accuracy with Scaling

Here, we introduce one approach to improve prediction accuracy, which is scaling. In this model, five explanatory variables are used: `age`, `fnlwgt`, `education-num`, `capital-gain`, and `capital-loss`. However, each of these variables has different units and magnitudes. If left as is, the model's learning may be biased by variables with larger values, diminishing the influence of those with smaller values.

To prevent this, we perform standardization of the explanatory variables. Standardization is a type of scaling in which the mean of each variable is subtracted from the data values, and the result is divided by the standard deviation. This removes the units between variables, and the magnitudes of the values align in meaning (a value of 0 means it is the mean, and a value of 1 means it is one standard deviation greater than the mean). To standardize the data, we use the `StandardScaler` class.

```
# Importing the class for standardization
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Setting X and y
X = adult[['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss']]
y = adult['fin_flg']

# Splitting into training and test data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random

# Standardization process
sc = StandardScaler()
sc.fit(X_train)
X_train_std = sc.transform(X_train)
X_test_std = sc.transform(X_test)

# Initializing and training the logistic regression class
model = LogisticRegression()
model.fit(X_train_std, y_train)

# Displaying the accuracy
print('Accuracy (train): {:.3f}'.format(model.score(X_train_std, y_train)))
print('Accuracy (test): {:.3f}'.format(model.score(X_test_std, y_test)))
```

```
Accuracy (train):0.811
Accuracy (test):0.810
```

As can be seen from the above results, the accuracy has improved compared to when no standardization was applied. By aligning the scales of the explanatory variables, machine learning algorithms can perform better. One point to keep in mind during standardization is that only the training data is used to calculate the mean and standard deviation. Test data is considered as unknown data that will be obtained in the future, so it cannot be used to evaluate the model.

> <Practice Question 5-3>

Use the `load_breast_cancer` function from the `sklearn.datasets` module to load the breast cancer dataset. Use `cancer.target` as the target variable and `cancer.data` as the explanatory variables to build a logistic regression prediction model. Use `train_test_split(random_state=0)` to split the data into training and test sets, and then calculate the score on the test data.

↳ 1 cell hidden

> <Practice Question 5-4>

Using the same settings as in <Practice Question 5-3>, stanardize the features of the same dataset and build the model. Then, compare the results with those from the previous question.

↳ 2 cells hidden

✓ 5.4 Decision Trees

Keywords: Decision Tree, Impurity, Entropy, Information Gain

In this section, we will learn how to build models using **decision trees**. A decision tree is a method of classifying data by repeatedly branching based on conditions of each attribute to reach a certain goal. When the target variable is categorical, it is called a **classification tree**; when the target is numerical, it is called a **regression tree**.

✓ 5.4.1 Mushroom Dataset

As an example for decision trees, we will use the mushroom dataset, which can be obtained from the following URL. The dataset contains information about mushrooms, with two categories: poisonous mushrooms and edible mushrooms.

<http://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data>

The goal is to determine whether a given mushroom is poisonous or not. The features of the mushrooms include shape, smell, gill size and color, among more than 20 different attributes. Using these features, conditions such as whether the cap is conical or not, whether the gill color is black or red, and whether its size is large or small can be used to classify the mushroom as poisonous or edible. Since the target variable in this case is categorical, it is an example of a classification tree.

This is how decision trees work: by repeatedly making decisions based on different attributes of the data, the tree divides the data into categories (e.g., poisonous or edible mushrooms). Various routes can lead to the target (such as identifying poisonous mushrooms), and these routes are represented in a tree structure, which is why it's called a decision tree.

First, let's load the mushroom dataset and display the first few rows to examine the data.


```
# Retrieve data
url = 'http://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaric
res = requests.get(url).content

# Read the retrieved data into a DataFrame object
mushroom = pd.read_csv(io.StringIO(res.decode('utf-8')), header=None)

# Set column labels
mushroom.columns = ['classes', 'cap_shape', 'cap_surface', 'cap_color', 'bruise
                    'gill_attachment', 'gill_spacing', 'gill_size', 'gill
                    'stalk_root', 'stalk_surface_above_ring', 'stalk_sur
                    'stalk_color_above_ring', 'stalk_color_below_ring',
                    'ring_number', 'ring_type', 'spore_print_color', 'po

# Display the first 5 rows
mushroom.head()
```

	classes	cap_shape	cap_surface	cap_color	bruises	odor	gill_attachment	g
0	p	x	s	n	t	p		f
1	e	x	s	y	t	a		f
2	e	b	s	w	t	l		f
3	p	x	y	w	t	p		f
4	e	x	s	g	f	n		f

5 rows × 23 columns

The contents of each variable are as follows.

1. cap-shape: bell=b, conical=c, convex=x, flat=f, knobbed=k, sunken=s
2. cap-surface: fibrous=f, grooves=g, scaly=y, smooth=s
3. cap-color: brown=n, buff=b, cinnamon=c, gray=g, green=r, pink=p, purple=u, red=e, white=w, yellow=y
4. bruises?: bruises=t, no=f
5. odor: almond=a, anise=l, creosote=c, fishy=y, foul=f, musty=m, none=n, pungent=p, spicy=s
6. gill-attachment: attached=a, descending=d, free=f, notched=n
7. gill-spacing: close=c, crowded=w, distant=d
8. gill-size: broad=b, narrow=n
9. gill-color: black=k, brown=n, buff=b, chocolate=h, gray=g, green=r, orange=o, pink=p, purple=u, red=e,

```

white=w,yellow=y
10. stalk-shape:      enlarging=e,tapering=t
11. stalk-root:      bulbous=b,club=c,cup=u,equal=e,
                    rhizomorphs=z,rooted=r,missing=?
12. stalk-surface-above-ring: fibrous=f,scaly=y,silky=k,smooth=s
13. stalk-surface-below-ring: fibrous=f,scaly=y,silky=k,smooth=s
14. stalk-color-above-ring:  brown=n,buff=b,cinnamon=c,gray=g,orange=o,
                    pink=p,red=e,white=w,yellow=y
15. stalk-color-below-ring:  brown=n,buff=b,cinnamon=c,gray=g,orange=o,
                    pink=p,red=e,white=w,yellow=y
16. veil-type:          partial=p,universal=u
17. veil-color:         brown=n,orange=o,white=w,yellow=y
18. ring-number:        none=n,one=o,two=t
19. ring-type:          cobwebby=c,evanescent=e,flaring=f,large=l,
                    none=n,pendant=p,sheathing=s,zone=z
20. spore-print-color:    black=k,brown=n,buff=b,chocolate=h,green=r,
                    orange=o,purple=u,white=w,yellow=y
21. population:         abundant=a,clustered=c,numerous=n,
                    scattered=s,several=v,solitary=y
22. habitat:            grasses=g,leaves=l,meadows=m,paths=p,
                    urban=u,waste=w,woods=d

```

The target variable is `classes`. When it is `p`, it indicates a poisonous mushroom, and when it is `e`, it indicates an edible mushroom. Each row represents the information of one mushroom, with attributes like `cap_shape` and `cap_surface` attached to it. For example, the first row represents a mushroom with classes as `p`, meaning it's poisonous, and `cap_shape` as `x` (`convex` or mound-shaped).

By running the following program, it can be confirmed that the dataset consists of 8124 rows and 23 columns, and there are no missing values.

```

print('Data shape: {}'.format(mushroom.shape))
print('Number of missing values: {}'.format(mushroom.isnull().sum().sum()))

```

```

Data shape: (8124, 23)
Number of missing values: 0

```

✓ 5.4.2 Data Organization

There are many explanatory variables, but to simplify the discussion, we will limit the explanatory variables to four: `gill_color`, `gill_attachment`, `odor`, and

`cap_color`. As shown above, these variables are categorical, for example, `gill_color` is represented as 'k' for `black` and 'n' for `brown`. Decision trees require both explanatory and target variables to be numerical. Therefore, categorical variables must be converted into numerical variables.

To achieve this, we will use dummy encoding. Dummy encoding means converting categorical variables, such as a gender variable with values `male` or `female`, into two separate columns: a 'male' column and a 'female' column. Specifically, if the gender is `male`, the 'male' column will be 1, and the `female` column will be 0 (this is also known as `one-hot` encoding). You can use the `get_dummies` function from pandas to create dummy variables.

```
mushroom_dummy = pd.get_dummies(mushroom[['gill_color', 'gill_attachment', 'odc
mushroom_dummy.head()
```

	<code>gill_color_b</code>	<code>gill_color_e</code>	<code>gill_color_g</code>	<code>gill_color_h</code>	<code>gill_color_k</code>	<code>gill_color_n</code>
0	0	0	0	0	1	0
1	0	0	0	0	1	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	1	0

5 rows × 33 columns

As described above, in the transformed data, each new variable represents a combination of the original variable name and its value. For example, if `gill_color_k` is set to 1, it indicates that `gill_color` was `k`. Dummy encoding is the simplest way to flag (or quantify) categorical variables in this manner.

Next, we'll also convert the target variable `classes` into a new variable `flg`. This step is necessary because even target variables representing categories must be in numerical form for the input data. The process converts values in the `classes` variable to 1 if they are p and 0 otherwise (using the `lambda` function), adding this new variable as `flg`. By using the `map` function, this transformation is applied to all elements (cells). With the target variable now expressed as 0/1 numerical values and the categorical feature variables dummy-encoded, the data is ready for input into the decision tree algorithm.

```
# Convert target variable to flag (0/1)
mushroom_dummy['flg'] = mushroom['classes'].map(lambda x: 1 if x == 'p' else 0)
```

✓ 5.4.3 Entropy: A Measure of Impurity

Before constructing a decision tree model, let's examine how decision trees are formed from the perspective of **impurity** in category classification. Impurity represents the state of differentiation (e.g., distinguishing between poisonous and edible mushrooms). High impurity means the categories are not well separated. For example, if we divide the data based on whether **cap_color** is "c" (using TRUE (1) or FALSE (0)), we can cross-tabulate to see how many poisonous mushrooms fall into each group.

The table below shows a cross-tabulation with rows representing whether **cap_color** is "c" (1 for true, 0 for false), and columns representing the poison flag **flg** (1 for poisonous, 0 for non-poisonous).

```
mushroom_dummy.groupby(['cap_color_c', 'flg'])['flg'].count().unstack()
```

	flg	
	0	1
cap_color_c		
False	4176	3904
True	32	12

From the table above, we see that when **cap_color** is "c" (**cap_color_c** is 1), there are 12 poisonous mushrooms (**flg** is 1) and 32 non-poisonous mushrooms (**flg** is 0). Conversely, when **cap_color** is not "c" (**cap_color_c** is 0), there are 3,904 poisonous mushrooms and 4,176 non-poisonous mushrooms.

This result suggests that the information about whether **cap_color** is "c" or not isn't very useful for distinguishing poisonous mushrooms, as both categories contain a relatively consistent proportion of poisonous mushrooms.

On the other hand, if we split the data based on whether another variable, **gill_color**, is "b" (TRUE (1) or FALSE (0)), the cross-tabulation result is as follows

```
mushroom_dummy.groupby(['gill_color_b', 'flg'])['flg'].count().unstack()
```

flg	0	1
gill_color_b		
False	4208.0	2188.0
True	NaN	1728.0

Here, we examined two variables (`cap_color_c` and `gill_color_b`) as examples, but there are many other variables, each of which could be split in a similar way. Decision trees are algorithms that assess which of these variables provides the most useful split for classification, using impurity as a criterion for determining the quality of each split. One of the most commonly used measures of impurity is **entropy**. The entropy for a dataset S is defined by the following formula $H(S)$, where S is the data set, n is the number of categories, and p_i represents the proportion of data samples in each category.

$$H(S) = - \sum_{i=1}^n (p_i \log_2 p_i)$$

In this example, there are two categories (poisonous or edible), with the proportion of edible mushrooms represented by p_1 and the proportion of poisonous mushrooms by p_2 . Let's consider a branching condition in which both poisonous and edible mushrooms are equally represented. In this case, $p_1 = p_2 = 0.5$, and entropy can be calculated using the formula above. We will use a base-2 logarithm function (`np.log2`).

```
- (0.5 * np.log2(0.5) + 0.5 * np.log2(0.5))
```

```
1.0
```

As shown above, we can confirm that entropy reaches 1.0. This actually represents the maximum level of randomness in the data. Since the dataset contains an equal proportion (0.5) of both poisonous and non-poisonous mushrooms, it indicates a completely unidentifiable state. Now, let's consider the case where the proportion of non-poisonous mushrooms p_1 is 0.001, and the proportion of poisonous mushrooms p_2 is 0.999.

```
- (0.001 * np.log2(0.001) + 0.999 * np.log2(0.999))
```

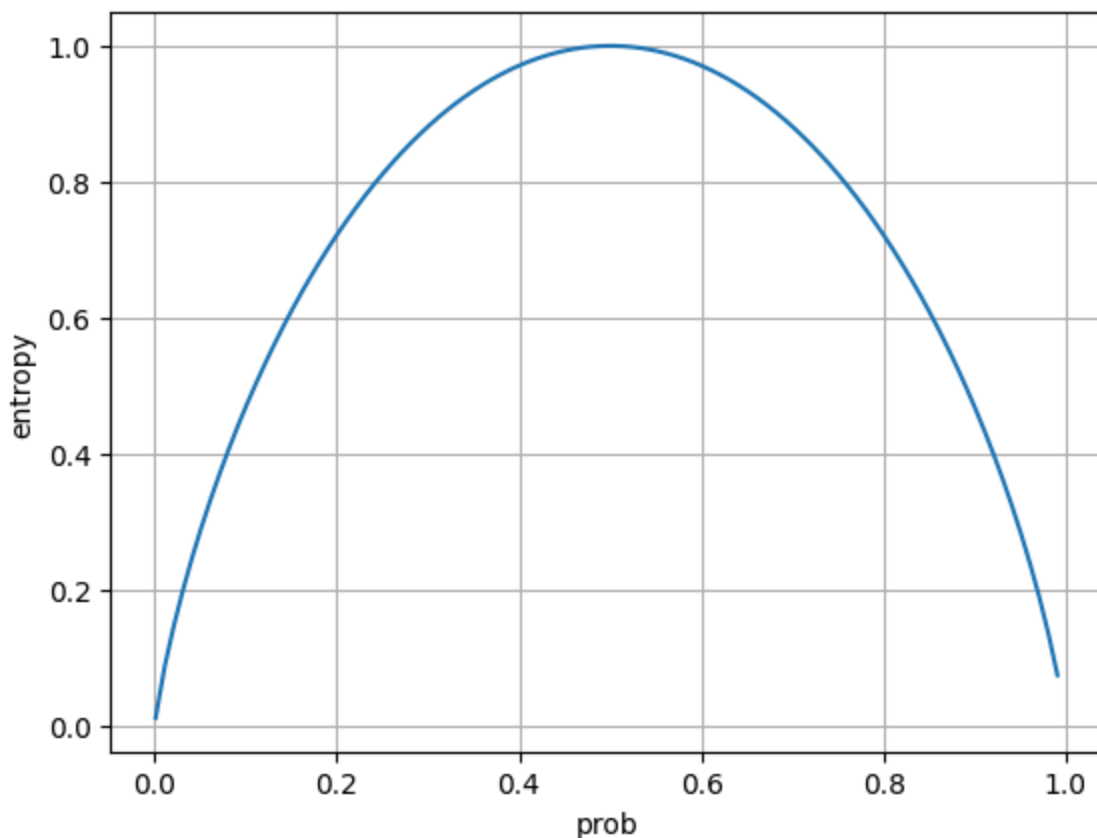
```
0.011407757737461138
```

As shown above, the entropy is close to 0. This reflects a state where the data is almost completely identified as poisonous mushrooms, thus resulting in low entropy. In summary, entropy values near 1.0 indicate an unidentifiable state, whereas values close to 0.0 suggest a well-identifiable state. In this example, since we are dealing with a binary classification, the categories follow the relationship $p_1 = 1 - p_2$, allowing us to express the entropy formula as follows.

```
def calc_entropy(p):  
    return - (p * np.log2(p) + (1 - p) * np.log2(1 - p) )
```

Since p is a probability, it ranges between 0 and 1. When we plot entropy as a function of p using the entropy formula, we get a graph like the one below. This confirms that entropy reaches a maximum value of 1 and a minimum of 0.

```
# Vary p values from 0.001 to 0.999 in 0.01 increments  
p = np.arange(0.001, 0.999, 0.01)  
  
# Plot the graph  
plt.plot(p, calc_entropy(p))  
plt.xlabel('prob')  
plt.ylabel('entropy')  
plt.grid(True)
```



In the explanation so far, we described how entropy represents classification impurity. Now, let's calculate the entropy for the mushroom dataset. The dataset we're using contains a total of 8,124 rows. Since the target variable `flg` represents categories, we will count the values in this data.

```
mushroom_dummy.groupby('flg')['flg'].count()
```

```

      flg
flg
0    4208
1    3916

dtype: int64

```

From the above, we can see that there are 4,208 non-poisonous mushrooms (0) and 3,916 poisonous mushrooms (1). Therefore, the proportion of non-poisonous mushrooms is 0.518 (= 4208/8124), and the proportion of poisonous mushrooms is 0.482 (= 3916/8124). Thus, the initial entropy value is 0.999, as shown below.

```
entropy_init = - (0.518 * np.log2(0.518) + 0.482 * np.log2(0.482))
print('Initial entropy of hte poisonous mushroom data: {:.3f}'.format(entropy_i
```

```
Initial entropy of hte poisonous mushroom data: 0.999
```

✓ 5.4.4 Information Gain: Evaluating Split Value

Entropy is closer to 1 when identification is poor and closer to 0 when identification is accurate. The next concept to consider is which explanatory variable should be used for branching to reduce impurity (which, in the mushroom data, starts at 0.999). The concept to grasp here is **information gain**. Information gain is an indicator that shows how much entropy has decreased before and after a data split using a particular variable. As before, we will use the two variables, `cap_color_c` and `gill_color_b`, to show which of them is more useful as a split condition using information gain. First, we will split the data into two groups based on whether `cap_color` is `c` or not and calculate the entropy for each group based on the proportion of poisonous mushrooms.

```
mushroom_dummy.groupby(['cap_color_c', 'flg'])['flg'].count().unstack()
```

flg	0	1
cap_color_c		
False	4176	3904
True	32	12

```
# Entropy when cap_color is not `c`
p1 = 4176 / (4176 + 3904)
p2 = 1 - p1
entropy_c0 = - (p1 * np.log2(p1) + p2 * np.log2(p2))
print('entropy_c0: {:.3f}'.format(entropy_c0))
```

```
entropy_c0: 0.999
```

```
# Entropy when cap_color is 'c'
p1 = 32 / (32 + 12)
p2 = 1 - p1
entropy_c1 = - (p1 * np.log2(p1) + p2 * np.log2(p2))
print('entropy_c1: {:.3f}'.format(entropy_c1))
```

```
entropy_c1: 0.845
```

The overall entropy before the split was 0.999. In this context, if we refer to the data before the split as the parent dataset and the data after the split as the child datasets, information gain is defined as: "**Parent dataset entropy - $\sum$ {(child dataset size / parent dataset size) $\times$ child dataset entropy}**". The larger this value, the greater the reduction in entropy between the parent and child datasets, indicating a more useful splitting condition. Now, let's calculate the part of $\sum$ {(child dataset size / parent dataset size) $\times$ child dataset entropy} as follows.

```
entropy_after = (4176 + 3904) / 8124 * entropy_c0 + (32 + 12) / 8124 * entropy_c1
print('Average entropy after data split: {:.3f}'.format(entropy_after))
```

```
Average entropy after data split: 0.998
```

This result shows that the information gain, calculated as the difference in entropy before and after the data split, is 0.001. This indicates that the entropy has not decreased significantly, and we can quantitatively conclude that whether cap_color is c or not is not a very useful splitting condition.

```
print('Information gain from splitting the variable cap_color: {:.3f}'.format(entropy_c0 - entropy_after))
```


Information gain from splitting the variable cap_color: 0.001

On the other hand, when calculating the information gain from whether `gill_color` is `b`, it is 0.269, as shown below. This shows that it is a more beneficial branching condition, as it reduces entropy more significantly than the previous condition. One thing to note here is the calculation of entropy when `gill_color` is `b`. The definition of entropy strictly requires the calculation to be done for non-empty categories. Since there are no samples where the `flg` variable is 0 when `gill_color` is `b`, we do not include the term $p_1 \times \log_2(p_1)$ in the sum for the entropy calculation.

```
mushroom_dummy.groupby(['gill_color_b', 'flg'])['flg'].count().unstack()
```

	flg	0	1
gill_color_b			
False		4208.0	2188.0
True		NaN	1728.0

```
# Entropy when gill_color is not b
p1 = 4208 / (4208 + 2188)
p2 = 1 - p1
entropy_b0 = - (p1 * np.log2(p1) + p2 * np.log2(p2))

# Entropy when gill_color is b
p1 = 0 / (0 + 1728)
p2 = 1 - p1
entropy_b1 = - (p2 * np.log2(p2))

entropy_after = (4208 + 2188) / 8124 * entropy_b0 + (0 + 1728) / 8124 * entropy_b1
print('Information gain from splitting by gill_color: {:.3f}'.format(entropy_ir
```

Information gain from splitting by gill_color: 0.269

With the above, we have reviewed the decision tree generation process (how to determine the effectiveness of condition splits). A decision tree splits the data based on the splitting condition with the highest information gain and continues to explore splitting conditions that maximize information gain at each subsequent level. We have introduced entropy as a measure of impurity, but other measures, such as **Gini impurity** and classification error, also exist. Gini impurity is related to the Gini coefficient, which

was discussed in a comprehensive problem related to probability and statistics. This lecture will omit further details, but if you are interested, please feel free to look it up.

[Let's Try]

Let's look into Gini impurity and classification error (misclassification rate). What kind of indicators are they? Also, how can they be incorporated when constructing a decision tree model?

As mentioned in the previous section (5.4 Regularized Regression: Lasso and Ridge Regression), model complexity was discussed, but in the case of decision trees, model complexity is determined by the number of splits. Remember that allowing more splits results in a more complex model.

✓ 5.4.5 Building a Decision Tree Model

Now that we understand how decision trees work, let's proceed with building a decision tree model. By using the `DecisionTreeClassifier` class from the `sklearn.tree` module, we can construct a decision tree model. In the program below, when using the `DecisionTreeClassifier` class, the parameter `criterion` is set to `entropy`, which specifies entropy as the criterion for the splitting condition.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

# Data split
X = mushroom_dummy.drop('flg', axis=1)
y = mushroom_dummy['flg']
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

# Initialize and train the decision tree classifier
model = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=0)
model.fit(X_train, y_train)

print('Accuracy(train): {:.3f}'.format(model.score(X_train, y_train)))
print('Accuracy(test): {:.3f}'.format(model.score(X_test, y_test)))
```

Accuracy(train):0.991
Accuracy(test):0.992

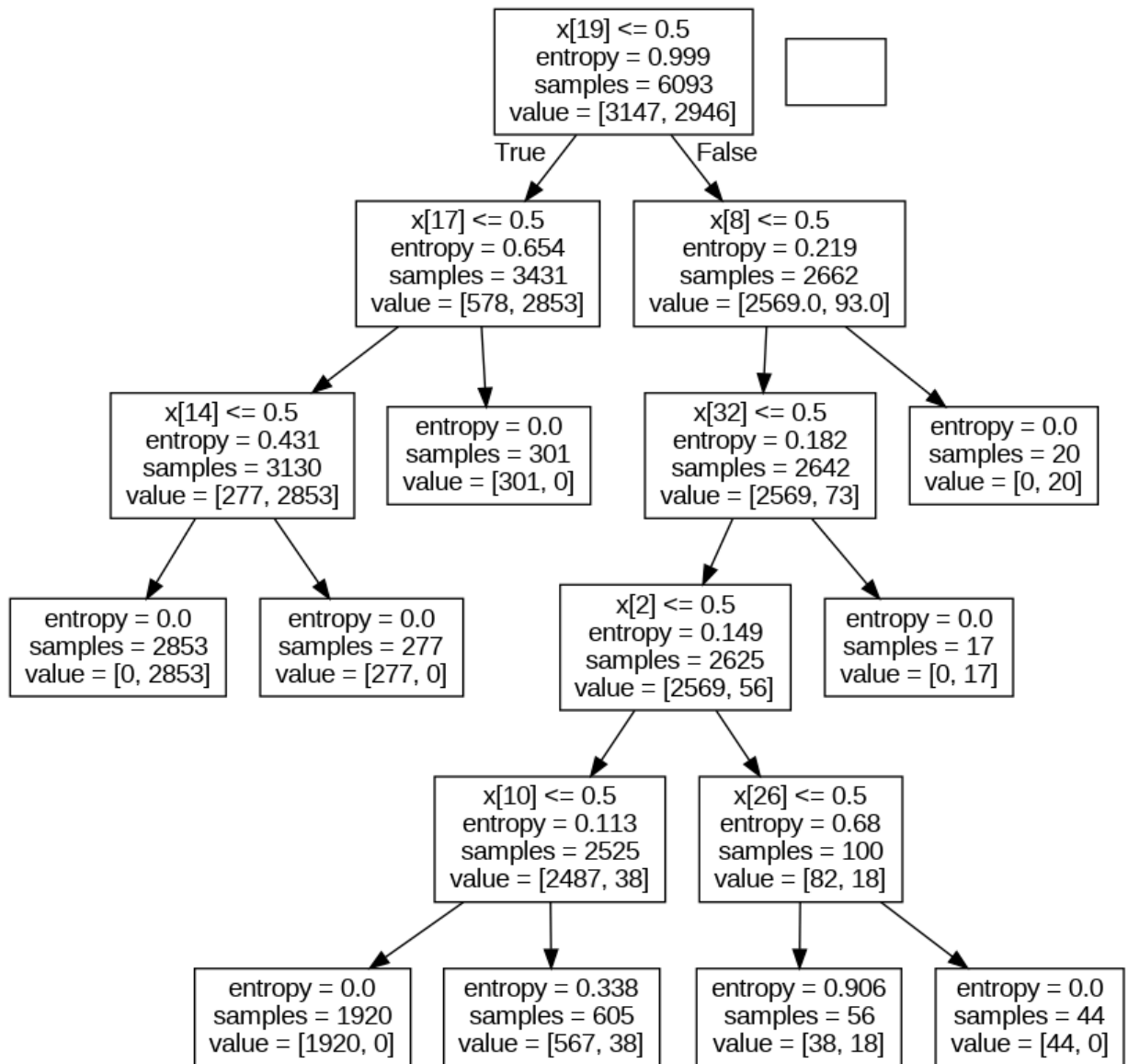
The result shows an accuracy of about 89% on the test data. The `max_depth` parameter controls the depth of the decision tree, set to 5 in the above example. Increasing the depth allows for more branching conditions. If you want a more complex model to improve accuracy, you can create a deeper tree. (However, be cautious, as making the tree too deep can increase the risk of overfitting.) Additionally, with decision trees, there's no need to perform standardization—a step essential for many other models—when building the model, as it doesn't affect the results.

As a reference, you can visualize the results of a decision tree as shown below. (To run this program, you'll need to install the `pydotplus` and `graphviz` packages, but due to the complexity of environment setup, these steps are omitted in this lecture.)

Looking at the visualization results below, you can see that repeated branching forms a binary tree structure. The rectangular nodes in the tree are read from the top down. The top variable (`X[0]`), which is the first column of explanatory variables, (`gill_color_b`) moves to the right (`False`) when greater than 0.5. The sample count for this subset is 1,302, and the entropy is zero. This means that when the (`gill_color_b`) flag is 1 (`X[0] <= 0.5`), the mushroom is identified as poisonous based on this branch.

```
# Reference program
# You need to install pydotplus and graphviz
from sklearn import tree
import pydotplus
from six import StringIO
from IPython.display import Image

dot_data = StringIO()
tree.export_graphviz(model, out_file=dot_data)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
Image(graph.create_png())
```



The reference book "Strategic Data Science Introduction: Concepts and Techniques for Business" is a recommended resource used to explain decision trees in this section. This book was mentioned in other topics as well, and it is recommended for its clear explanations.

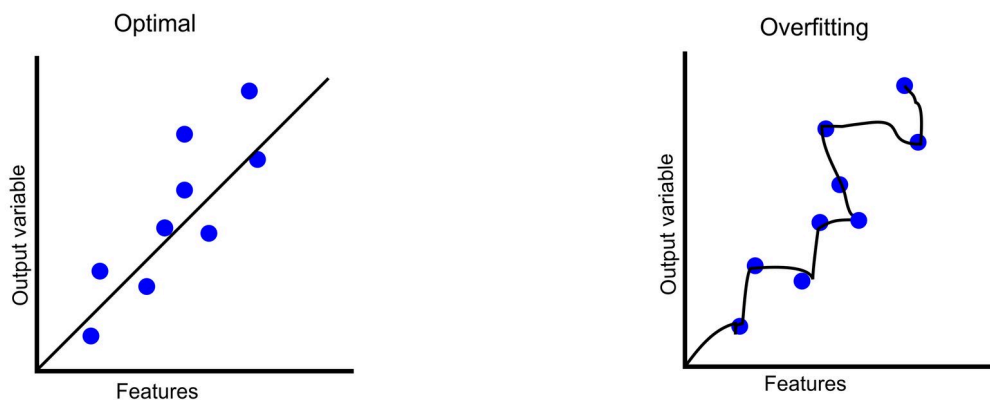
> <Practice Question 5-5>

Using the `load_breast_cancer` function from the `sklearn.datasets` module, load the breast cancer data. Set `cancer.target` as the target variable and `cancer.data` as the explanatory variable, then build a decision tree model and check the training and test scores. Try adjusting parameters, such as the depth of the tree, and compare the results.

↳ 1 cell hidden

✓ 5.5 Overfitting

Overfitting happens when a model clings too tightly to training data, fitting noise rather than true patterns. This leads to high accuracy on training data but poor performance on unseen data.



Reference: <https://www.freecodecamp.org/news/what-is-overfitting-machine-learning/>

Overfitting can be mitigated through various techniques. Below are some key considerations:

1. Increasing training data
2. Cross-validation
3. Early stopping
4. Regularization (we will discuss this further in the section 5.5)

✓ 5.6 Regularized Regression: Lasso and Ridge Regression

Keywords: Regularization, Lasso Regression, Ridge Regression

Next, I will explain Lasso regression and Ridge regression. These techniques have the characteristic of preventing overfitting in multiple regression models, especially in situations where small changes in the input lead to large changes in the output.

✓ 5.6.1 Features of Lasso Regression and Ridge Regression

In multiple regression, the regression coefficients are estimated to minimize the squared error between the predicted values and the target variable. In contrast, Lasso regression and Ridge regression add mechanisms that avoid making the regression coefficients too large, in addition to minimizing the squared error. Generally, models with large regression coefficients tend to produce large changes in the output with small changes in the input. This leads to a complex input-output relationship. Such complex models may fit the training data well but are at a higher risk of overfitting and not generalizing well to new, unseen data. To address this, Lasso and Ridge regression add a term that represents the complexity of the model to the loss function, and they estimate the regression coefficients by minimizing both the error and the complexity term.

Specifically, in Lasso and Ridge regression, the loss function for estimating the regression coefficients is defined as follows. The second term in this equation is called the regularization term. When $q=1$, it is called **Lasso regression**, and when $q=2$, it is called **Ridge regression** (M : the number of variables, w : weights or coefficients, λ : regularization parameter). The regularization term serves the purpose of reducing the model's complexity. **Regularization** refers to techniques that generally aim to reduce the complexity of a model.

$$\begin{array}{l} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \sum_{j=1}^M |w_j|^q \\ \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \sum_{j=1}^M w_j^2 \end{array}$$

From the definition of this equation, it can be understood that as the number of variables M increases or the weights increase, the value of the second term in the loss

function becomes larger, which serves as a penalty. In contrast to multiple regression or logistic regression, where the analyst adjusts the number of explanatory variables to control the model's complexity, Lasso and Ridge regression adjust the complexity of the model by keeping the size of the parameters themselves small. If there is a discrepancy between the training score and test score, using an algorithm with a regularization term may improve the model's generalization performance.

Note that Scikit-learn's logistic regression includes a regularization term with $q=2$ in the loss function by default.

✓ 5.6.2 Comparison of Multiple Regression and Ridge Regression

Let's use the automobile price data (`auto`) from the previous multiple regression example to create a Ridge regression model and check the difference between the results of multiple regression and Ridge regression. Here, we will display the first 5 rows of the data used.

```
auto.head()
```

	price	horsepower	width	height
0	13495	111	64.1	48.8
1	16500	111	64.1	48.8
2	16500	154	65.5	52.4
3	13950	102	66.2	54.3
4	17450	115	66.4	54.3

Using the `Ridge` class from the `sklearn.linear_model` module, you can construct a Ridge regression model. The following program creates both a multiple regression model (`linear`) using the `LinearRegression` class and a Ridge regression model (`ridge`) using the `Ridge` class, then compares their results.

```
# Ridge Regression Class
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split

# Split into training and testing data
X = auto.drop('price', axis=1)
y = auto['price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random
```

```
# Model construction and evaluation
linear = LinearRegression()
ridge = Ridge(random_state=0)

for model in [linear, ridge]:
    model.fit(X_train, y_train)
    print('{} (train): {:.6f}'.format(model.__class__.__name__, model.score(X_train, y_train)))
    print('{} (test): {:.6f}'.format(model.__class__.__name__, model.score(X_test, y_test)))
```

LinearRegression (train):0.733358
 LinearRegression (test):0.737069
 Ridge (train):0.733355
 Ridge (test):0.737768

Both have very similar performance, but as a trend, the linear regression model has a higher accuracy on the training data, while the Ridge regression model outperforms on the test data. This reversal is likely due to the effect of the regularization term.

> <Practice Question 5-6>

Evaluate the Lasso regression on the data used in Practice Question 5-1. Use the `Lasso` class from the `sklearn_linear` model module. The Lasso class has parameters that can be configured, so please look into it. Specifically, refer to the official documentation below:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html#sklearn.linear_model.Lasso

↳ 1 cell hidden

✓ 5.7 k-Nearest Neighbors (k-NN)

Keywords: k-NN, lazy learning, memory-based learning

In this section, we'll learn about **k-NN (k-Nearest Neighbor)**. For example, consider groups A and B with known attributes for each member. Now, imagine a new person arrives, and we don't know which group they belong to.

To determine this, we use k-NN by selecting k individuals with the most similar attributes to this new person. We check whether more of these k individuals belong to group A or group B, and assign the new person to the group with the majority. Here, k represents

the number of people used in the decision. k-NN is also called lazy learning or memory-based learning, as it memorizes the training data directly without explicit training.

✓ 5.7.1 Building a k-NN Model

Here's how you can build a k-NN model using the `KNeighborsClassifier` from the `sklearn.neighbors` module, with the breast cancer dataset as an example. The dataset can be loaded using the `load_breast_cancer` function.

In this example, we will vary the value of k from 1 to 20 and observe how the accuracy of the training and testing data changes. When k is small, the training and testing accuracy will differ significantly. However, around k = 6 to 8, the training and testing accuracy will become more aligned. Further increasing k does not result in substantial improvements in model accuracy. Therefore, there is no need to use a very large k if accuracy doesn't improve much. In this case, setting k between 6 and 8 seems optimal.

Note that this is an example for classification tasks. For regression tasks, you would use the `KNeighborsRegressor` class.

```
# Importing the libraries for data and model construction
from sklearn.datasets import load_breast_cancer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

# Loading the dataset
cancer = load_breast_cancer()

# Splitting the dataset into training and test data
# Stratify for stratified sampling
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify = cancer.target, random_state=0)

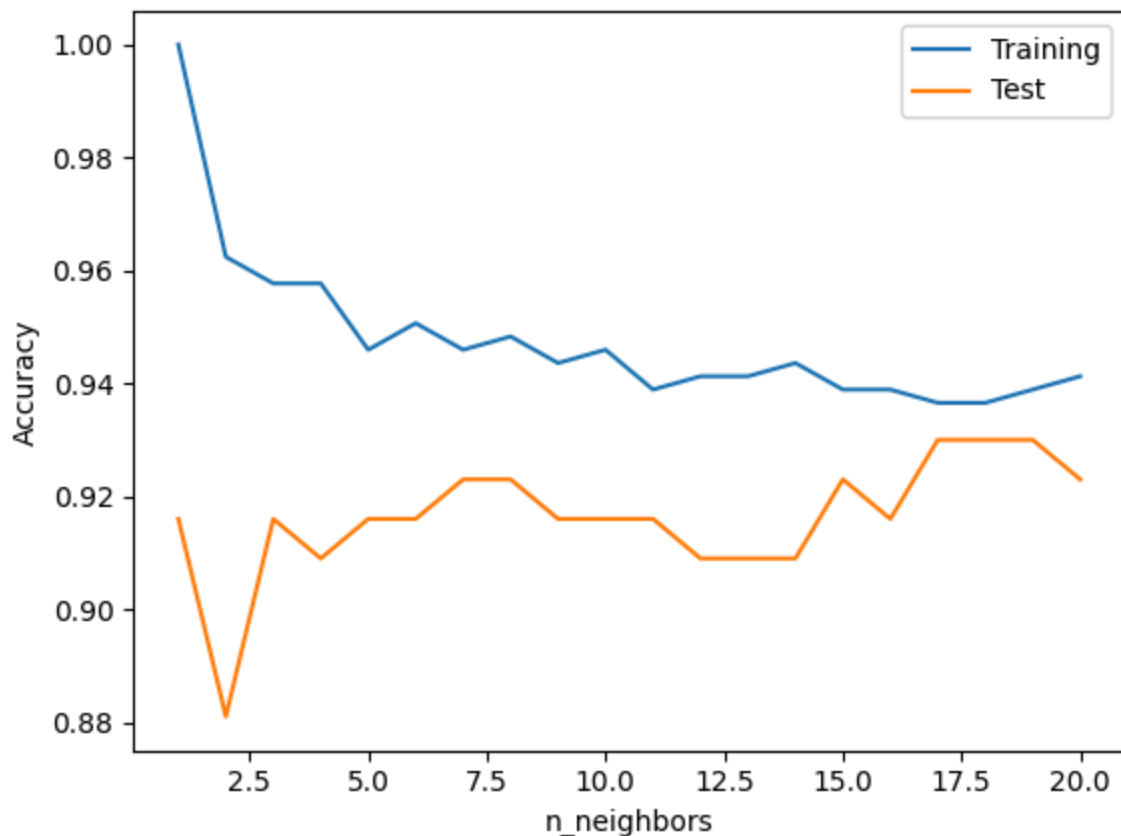
# Preparing lists to store accuracy for training and testing
training_accuracy = []
test_accuracy = []

# Training the model
for n_neighbors in range(1, 21):
    model = KNeighborsClassifier(n_neighbors=n_neighbors)
    model.fit(X_train, y_train)
    training_accuracy.append(model.score(X_train, y_train))
    test_accuracy.append(model.score(X_test, y_test))

# Plotting the accuracy graph
plt.plot(range(1, 21), training_accuracy, label='Training')
plt.plot(range(1, 21), test_accuracy, label='Test')
```

```
plt.ylabel('Accuracy')  
plt.xlabel('n_neighbors')  
plt.legend()
```

<matplotlib.legend.Legend at 0x7cedd939d600>



[Let's Try]

Investigate how k-NN regression is calculated.

> <Practice Question 5-7>

Construct and validate a k-NN model using the mushroom data we used in section 5.5 'Decision Trees.' Please execute it while changing the k parameter.

↳ 1 cell hidden

> <Practice Question 5-8>

Using the student test result data (student-mat.csv) we used earlier, define the target variable as G3 and the explanatory variables as X (using the students' attribute data). Change the k parameter of k-NN and consider which k is optimal.

Since the target variable is numeric and this is a regression task, please use `KNeighborsRegressor`. In regression, the output will be the average of the k nearest neighbors' values.

↳ 2 cells hidden

✓ 5.8 Support Vector Machines

Keywords: Support Vectors, Margin

Support Vector Machine (SVM) is a method that draws a boundary between categories with a line that has a margin, rather than just a single line. For example, when drawing a boundary line to separate two groups, there are various ways to draw the line. However, SVM draws the line in such a way that the distance (margin) between the closest points of each group (the support vectors) and the boundary line is maximized.

✓ 5.8.1 Building a Support Vector Machine Model

Let's build a model using Support Vector Machine. To do this, we'll use the `LinearSVC` class from the `sklearn.svm` module. For this example, we will use the same breast cancer dataset that was used in the k-NN model construction.

```
# Import the SVM library
from sklearn.svm import LinearSVC

# Library for splitting training and test data
from sklearn.model_selection import train_test_split

# Load the data
cancer = load_breast_cancer()

# Split into training and test data
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify = cancer.target, random_state=0)

# Initialize and train the model
model = LinearSVC()
model.fit(X_train, y_train)

# Print scores for training and test data
print('Accuracy (train): {:.3f}'.format(model.score(X_train, y_train)))
print('Accuracy (test): {:.3f}'.format(model.score(X_test, y_test)))
```

```
Accuracy (train):0.972  
Accuracy (test):0.937
```

With Support Vector Machines (SVMs), standardizing data can often improve the model's score. Trying this out shows noticeable improvement in performance.

```
# Load data  
cancer = load_breast_cancer()  
  
# Split data into training and test sets  
X_train, X_test, y_train, y_test = train_test_split(  
    cancer.data, cancer.target, stratify = cancer.target, random_state=0)  
  
# Standardize data  
sc = StandardScaler()  
sc.fit(X_train)  
X_train_std = sc.transform(X_train)  
X_test_std = sc.transform(X_test)  
  
# Initialize and train the model  
model = LinearSVC()  
model.fit(X_train_std, y_train)  
  
# Training and test scores  
print('Accuracy (train):{:.3f}'.format(model.score(X_train_std, y_train)))  
print('Accuracy (test):{:.3f}'.format(model.score(X_test_std, y_test)))
```

```
Accuracy (train):0.993  
Accuracy (test):0.951
```

[Let's Try]

Investigate which class can be used to build a model for regression (predicting continuous variables) with Support Vector Machines.

This concludes the explanation of various model-building methods for supervised learning. Let's review the flow of model-building for each method and the concept of evaluating machine learning models (evaluating with data not used for training).

✓ <Practice Question 5-9>

Using the breast cancer dataset, build a model to predict `cancer.target` with the `SVC` class from the `sklearn.svm` module. Set the model as `model =`

`SVC(kernel='rbf', random_state=0, C=2)`. After building the model, split the data into training and test sets, standardize them, and check the scores.

✓ 5.9 Comprehensive Questions

Comprehensive Question 5- 1 Terms in Supervised Learning (1)

Explain the roles and meanings of each term related to supervised learning. In what situations are they used? Use resources like the internet or reference materials to investigate the following terms:

- Regression
- Classification
- Supervised Learning